Breadth-First Search

Breadth-First Search (BFS) is a graph traversal algorithm that explores vertices in layers: all vertices at distance k are visited before any vertex at distance $k+1$.

Natural Explanation

Imagine an expanding ring from a stone dropped in a pond. BFS spreads outward level by level, visiting all neighbors at the current distance before moving further away.

Formal Definition

Given a graph $G = (V, E)$ and a starting vertex s:

BFS visits all vertices reachable from s by exploring level by level. All vertices at distance 1 are visited first, then distance 2, and so on.

Pseudocode (with Queue)

```
BFS(graph, start_vertex)
    visited = empty set
    queue = empty queue
    queue.ENQUEUE(start_vertex)
    add start_vertex to visited

    while not queue.ISEMPTY():
        vertex = queue.DEQUEUE()
        process(vertex)
        for each neighbor in graph.NEIGHBORS(vertex):
            if neighbor not in visited:
                add neighbor to visited
                queue.ENQUEUE(neighbor)
```

Complexity

- **Time**: $O(|V| + |E|)$ — visit each vertex once, examine each edge once
- **Space**: $O(|V|)$ — queue size at most $|V|$

Distance and Shortest Paths

BFS implicitly computes **shortest path distances** in unweighted graphs.

If you track the parent of each vertex as it's discovered:

```
distance[start] = 0
for each other vertex v:
    distance[v] = distance[parent[v]] + 1
```

Then `distance[v]` is the shortest path length from `start` to `v` in an unweighted graph.

Key Properties

Visits level by level: BFS tree has clear level structure where edges only go between adjacent or same levels (no "long jumps").

Discovers connected components: run BFS from each unvisited vertex.

Shortest path tree: in unweighted graphs, following parent pointers gives a shortest path from start to any vertex.

Common Applications

- **Shortest paths in unweighted graphs**: computes shortest path distances
- **Level-order traversal** of trees
- **Finding connected components**: how many separate regions are connected?
- **Checking bipartiteness**: color vertices with two colors such that neighbors differ; possible if and only if graph is bipartite
- **Peer-to-peer networks**: flood-fill operations
- **Social networks**: finding friends at distance k

Comparison with DFS

Aspect	BFS	DFS
Order	level by level	deep first; backtrack
Data structure	[[Queues|queue]]	[[Stacks|stack]]
Shortest path (unweighted)	Yes	No
Space (worst case)	$O(\|V\|)$ wide	$O(\|V\|)$ deep

Aspect	BFS	DFS
Common use	Shortest paths; components	Reachability; topological sort; SCC

Related

- [[Depth-First Search]]
- [[Graphs]]
- [[Queues]]
- [[Shortest Paths]]